

3.4 조건 (Condition)

블록코딩에서 사용빈도 매우 높은 블록으로 "만일 ~이라면" 과 "만일 ~이라면, 아니면~" 의 두 가지 블록이 있다. 용도는 우리의 코드가 순차적으로 실행해 나갈 때, 어떤 조건에 따라 실행을 할 수도 있고, 하지 않을 수도 있는 경우에 따른 동작을 만들기 위한 용도라 할 수 있다.

이전 장에서 만들어 오고 있는 계산기 프로그램에서 사용자로부터 입력된 임의의 두 값만을 더해주는 것 외에 사용자에게 더 나은 사용성을 주기 위해 덧셈 뿐 아닌 뺄셈도 가능하고, 둘 중 어떤 계산을 선택할지를 사용자의 선택권에 맡기는 더 나은 프로그램으로 발전시켜보자. 이처럼 소프트웨어를 만드는 일을 한번의 단회성의 제작으로 끝나는게 아니라, 계속 진화해나가면서 더 나은 프로그램으로 버전업이 되어 가는 것이 일반적이며 이를 소프트웨어의 생명주기라고 할 수 있다.

실행결과

당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기입니다

first 0

second 0



대답 0

첫 번째 숫자를 입력하세요.

first 0

second 0



13

✓

두 번째 숫자를 입력하세요.

first 13

second 0



12

✓

덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요.

first 13

second 12



-

✓

입력된 두 수의 차는 1입니다

first 13

second 12



대답 0

블록코딩(1)

A Scratch-style block coding interface for a calculator. The blocks are as follows:

- Start** (green flag clicked):
 - Say "당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기 입니다" for 2 seconds.
- 2 초 기다리기** (wait 2 seconds).
- Ask for first number** (say "첫 번째 숫자를 입력하세요." for 2 seconds).
- Get first input** (ask "first" for "답" and store in "first").
- Ask for second number** (say "두 번째 숫자를 입력하세요." for 2 seconds).
- Get second input** (ask "second" for "답" and store in "second").
- Ask for operator** (say "덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요." for 2 seconds).
- Conditional Logic** (if-then blocks):
 - If '+'**: Calculate $first + second$ and say "입력된 두 수의 합은 과(와) first 값 + second 값을 합치기 과(와) 입니다" for 2 seconds.
 - If '-'**: Calculate $first - second$ and say "입력된 두 수의 차는 과(와) first 값 - second 값을 합치기 과(와) 입니다" for 2 seconds.

엔트리-파이썬(1)

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | first = 0
6 | second = 0
7 |
8 | def when_start():
9 |     Entry.print("당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기 입니다")
10 |    Entry.wait_for_sec(2)
11 |
12 |    Entry.input("첫 번째 숫자를 입력하세요.")
13 |    first = Entry.answer()
14 |    Entry.input("두 번째 숫자를 입력하세요.")
15 |    second = Entry.answer()
16 |
17 |    Entry.input("덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요.")
18 |    if Entry.answer() == "+":
19 |        Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")
20 |    if Entry.answer() == "-":
21 |        Entry.print("입력된 두 수의 차는 " + (first - second) + " 입니다")
```

블록코딩(2)



엔트리-파이썬(2)

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | first = 0
6 | second = 0
7 |
8 | def when_start():
9 |     Entry.print("당신이 입력한 두 수의 덧셈 또는 뺄셈을 할 수 있는 계산기 입니다")
10 |    Entry.wait_for_sec(2)
11 |
12 |    Entry.input("첫 번째 숫자를 입력하세요.")
13 |    first = Entry.answer()
14 |    Entry.input("두 번째 숫자를 입력하세요.")
15 |    second = Entry.answer()
16 |
17 |    Entry.input("덧셈계산을 원하시면 '+'를 입력하고, 뺄셈계산을 원하시면 '-'를 입력하세요.")
18 |    if Entry.answer() == "+":
19 |        Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")
20 |    else:
21 |        if Entry.answer() == "-":
22 |            Entry.print("입력된 두 수의 차는 " + (first - second) + " 입니다")
```

기존 프로그램에서는 변수 3개(first, second, result)를 사용했다면, 이번 프로그램에서는 변수 2개(first, second)만 사용하도록 개수를 줄였는데, 줄이는 방법은 기존에는 두 값의 계산결과를 다시 result라는 변수에 담아서 그 변수에 담긴 값을 출력하는 식으로 했다면, 이번에는 해당 변수 없이 (first + second), (first - second)라고 계산식을 괄호와 함께 사용함으로써 출력함수(print함수) 안에서 직접계산하여 결과 출력하도록 하였다.

34 17번 라인을 살펴보자. 우리는 사용자에게 '+' 또는 '-' 둘 중에 하나를 선택해 입력는 선택권을 주도록 안내했기 때문에 사용자가 둘 중에 하나의 원하는 연산종류를 정상적으로 입력한다고 가정하고, 사용자가 선택한 연산종류에 따라 코드실행을 달리해야하기 때문에 조건문을 사용했는데 파이썬 언어에서 조건문을 사용하는 문법은 다음과 같다.

if 조건:

조건이 만족(또는 참)일 때 실행할 내용들
공백 4칸 또는 탭 1칸

34 위에 문법에 빗대어 18-19번 라인의 다음의 코드를 이해할 수 있는지 확인해 보자.

```
18 | if Entry.answer() == "+":  
19 |     Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")
```

이전 장에서 '==' 기호(연산자)의 의미가 무엇인지 설명했다. 좌우의 두 값이 같은지를 확인하기 위한 연산자(operator)로서 사용자가 입력한 값이 '+' 였는지를 확인하고 있다. 만약 사용자의 입력값이 '+' 라면 그렇다면, if 로 시작한 문장의 다음에 위치한 코드들이 실행될 것이다. 여기서 코드작성시 주의해야 할 것은 우리가 [Hello World 예제코드 분석](#)에서 알아본 바와 같이 if 문 다음에 의도적인 공백(일반적으로 키보드의 스페이스바 로 4 칸 또는 탭으로 1칸을 누름)을 만들어 매 라인마다 같은 크기만큼 공백을 둬으로써 if 조건이 만족에 경우의 수행할 코드덩어리의 시작과 끝이 어디 인지를 분간할 수 있게 해주어야 한다는 것이다. 따라서, 공백을 정확하게 입력해야 한다는 것을 잘 숙지해야 하겠다.

34 위에 조건문의 사용이 정확하게 이해되었다면, 그 다음 20-21번 라인의 코드는 당연히 쉽게 이해될 것이다.

마지막으로, 위에 예제코드를 블록코딩(1과 2), 텍스트코딩(1과 2)로 구분해 놓은 이유를 설명해야 할 것인데, 1과 2의 두 종류의 코드는 모두 우리가 목표한 동일한 기능을 하되, 다만, 구현방법에 있어서 약간 다른 방식으로 구현한 코드로서 지금 배운 조건문 외 한 가지 더 추가적인 조건문 사용문법을 설명하기 위한 의도가 있었다.

우리가 블록코딩에서 "만일 ~이라면" 이라는 블록의 텍스트코딩에서는 어떻게 표현하는지를 지금까지 배웠다면, "만일 ~이라면, 아니면~" 이란 블록의 사용법도 텍스트코딩에도 존재함을 미루어 짐작할 수 있는데, 이를 문법사용법은 다음과 같다.

if 조건:

조건이 만족(또는 참)일 때 실행할 내용들
공백 4칸 또는 탭 1칸

else :

조건이 만족하지 않을(또는 거짓) 때 실행할 내용들
공백 4칸 또는 탭 1칸

```
18 | if Entry.answer() == "+":  
19 |     Entry.print("입력된 두 수의 합은 " + (first + second) + " 입니다")  
20 | else:
```

```

21 |     if Entry.answer() == "-":
22 |         Entry.print("입력된 두 수의 차는 " + (first - second) + " 입니다")

```

위에 코드는 아래와 같이 블록코딩과 비교해보면 더 쉽게 이해할 수 있다.



다만, if~else~ 형식의 텍스트코딩의 조건문의 사용상에 있어서도 주의해야할 부분은 이전과 마찬가지로 각 코드의 라인마다 동일한 공백을 길이를 얼마만큼 유지해야만 하는지에 주의를 기울여 코딩해야 한다는 것이다. 블록코딩에서는 비주얼적으로 조건문에 시작과 끝이 명확하게 감싸져있는 방식으로 표현해서 우리가 직관적으로 이해가 가능했지만, 텍스트코딩에서는 텍스트로만 이를 표현해야하는 한계성 안에서 블록코딩 표현상 감싸져 있는 코드덩어리는 텍스트코딩에서는 매 라인마다 동일 공백량의 길이조절을 통해(들여쓰기를 통해) 이를 표현하고 있음을 이해할 수 있다.